WARLOCK / PROJECT GUIDE

Code, games, and evaluation

A technical overview of the React application, arcade modes, 1v1 rooms, JavaScript evaluation, build commands, and deployment workflow.

2	1v1	JS
Arcade game modes	Live coding battle	Challenge evaluation language

Executive summary

Warlock is a browser-based coding game built with React, Vite, Monaco Editor, and HTML canvas. Players can control a claw machine or a bowling game with JavaScript. A separate 1v1 mode connects two players over a Node WebSocket server and compares solutions to one of three challenge problems.

Current code evaluation

Dual-mode submissions run in a dedicated browser Web Worker. The worker executes a named JavaScript function against fixed cases, stops after two seconds, and returns the number passed. The interface reports an 80% correctness score plus a 20% heuristic efficiency score.

Repository review

This review reflects the source currently in the repository. The latest production build completed successfully with Vite. The build still reports a large Monaco-related bundle warning. Oxlint exits successfully but reports warnings about unused imports, refs during render, and hook dependencies.

01 / SYSTEM OVERVIEW

How the application fits together

The app is a single-page React frontend plus a small Node service. The frontend owns the interface and local game simulation. The server owns temporary room coordination and broadcasts updates over WebSockets.

Layer	Main files	Responsibilities
Browser UI	src/App.jsx, src/components/DualRoom.jsx	Mode selection, editor controls, room flow, feedback and scores.
Editor and game	MonacoEditor.jsx, GameCanvas.jsx, BowlingCanvas.jsx	JavaScript editing, canvas animation, game state and game APIs.
Challenge runner	dualChallenges.js, codeRunner.worker.js	Challenge definitions, test data, score heuristic and bounded worker execution.
Room service	server/dualServer.js	Room membership, challenge selection, code sharing, submissions and winner state.
Build and deploy	vite.config.js, render.yaml, package.json	Vite build/dev server and Render WebSocket service configuration.

Two main player journeys

Journey	Steps
Single player	Choose Claw Machine or Bowling; edit the supplied JavaScript manual; run it; the script repeatedly calls the selected game API as the canvas animates.
Double player	Create or join a room; connect a second player; start a shared random challenge; edit code while it syncs live; run tests; compare scores.

Frontend and backend are separate processes

`npm run build` creates static files in `dist`. It does not start a website server or the room server. `npm run dev` starts Vite for frontend development. `npm run dual-server` starts the Node HTTP and WebSocket service on port 3001.

02 / SINGLE-PLAYER ARCADE

The games are programmable

The single-player editor receives a manual as starter code. Clicking Run compiles that text as a function with a game API parameter. The canvas loop invokes the compiled function on every animation frame until the player runs another script or changes game.

Game	JavaScript API	What the game does

Claw Machine	`claw.move(speed)`
	`claw.moveTo(x, speed)`
	`claw.drop()`
	`claw.release()`
	Sensors: x, y, isIdle, hasPlushie, atChute, nearestPlushie.
		A 480 x 420 canvas simulation with a claw, plushies, a chute, hit/miss odds, score and sound effects. Aim, drop, carry a plushie to the chute and release.

Bowling	`ball.move(speed)`
	`ball.moveTo(x, speed)`
	`ball.setAngle(deg)`
	`ball.turn(deg)`
	`ball.roll(power)`
	Sensors: x, y, angle, isIdle, isRolling, pinsStanding, pinsDown.
		A canvas lane with a ball and ten pins. Code aims and rolls the ball; physics update position and pin state, and score/strike feedback is shown.

Single-player execution sequence

```
Editor source
    -> new Function('claw' or 'ball', source)
    -> store compiled function in a React ref
    -> requestAnimationFrame loop calls the function each frame
    -> game API mutates the canvas simulation state
```

> **Execution limitation**
>
> This single-player path runs user code on the browser main thread and calls it every frame. A long-running loop can freeze the interface. Runtime exceptions are swallowed in the animation loop, so syntax errors are visible but many runtime errors have little feedback. Use only code you trust.

Other frontend pieces

`src/App.jsx` controls the lobby and mode switching. `src/components/MonacoEditor.jsx` configures Monaco themes, language workers, keyboard shortcuts and controlled text updates. `src/index.css` and `src/App.css` provide the dark neon visual system and responsive layouts. `src/assets/` contains game illustrations and objects.

03 / MULTIPLAYER

Room flow and shared state

The server is a stateful Node process using the `ws` library. It exposes `/health` over HTTP and `/dual-ws` over WebSockets. Room data lives in Maps in process memory; no database is used.

Event	Effect
join	Validates a 4 to 8 character alphanumeric room code, attaches a player ID/name, and sends the room snapshot.
code:update	Stores up to 100 KB of code and broadcasts it to the other player.
round:start	Requires two connected players, picks challenge 1-3, starts a 120-second round and clears prior submissions.
round:submit	Stores passed/total/score and chooses a winner or draw after two submissions arrive.
round:expire	Marks the round expired when the client timer reaches zero.

Room lifecycle

```
Browser A  -- join/code/submit -->  Node WebSocket server
Browser A  <-- room:state/code:opponent --  Node server  <-- Browser B

Room code: 4-8 letters/numbers; generated as 6 characters
Room size: 2 players
Round: random challenge, 120 seconds
Cleanup: after both disconnect, room is removed after 5 minutes
```

Deployment behavior

Render runs the room server as a Web Service and checks `/health`. Configure the frontend build with `VITE_DUAL_SERVER_URL=wss://.onrender.com/dual-ws`. For same-network local play, leave it blank and open the Vite host from both devices; allow ports 5173 and 3001 through the host firewall.

Current multiplayer limits

A server restart clears rooms and submissions. Room codes and player IDs are not authentication. The server accepts the score sent by the browser rather than independently running the challenge, so a modified client can falsify a score. Room submission values are clamped, but correctness is not verified server-side.

04 / CODE EVALUATION

How dual-mode code is evaluated

The challenge editor currently supports JavaScript only. `QUESTION_BANK` defines three named functions and fixed input/expected-output cases. Submission runs the provided function once against each case in `codeRunner.worker.js`.

1. Submit

The app requires an active round and non-empty solution, sends the latest code to the room, then starts the local worker.

2. Resolve function

The worker evaluates the source and looks up the named function: `twoSum`, `reverseString`, or `isPalindrome`.

3. Execute cases

Each case gets a structured clone of its inputs. The function result is compared with expected output via JSON serialization. If it returns null/undefined, the first argument is used, which supports in-place `reverseString`.

4. Enforce timeout

The parent terminates the Web Worker after 2 seconds. Syntax/runtime errors return a message; failed runs remain editable and can be retried.

5. Score and share

The browser computes correctness plus heuristic efficiency and sends the result to the WebSocket server for comparison.

```
score = round((passed / total) * 80 + (efficiency / 100) * 20)

Examples with efficiency=100:
  4/4 cases -> 100
  3/4 cases -> 80

Examples with efficiency=80:
  4/4 cases -> 96
  3/4 cases -> 76
```

Challenge bank

Problem	Cases	Current efficiency hint
Two Sum	4	Awards 100 for `new Map`; otherwise 80.
Reverse String	4	The generic loop regex currently gives 80 for `while`, otherwise 100.
Palindrome Number	5	Awards 100 for `% 10` or `Math.floor`; otherwise 80.

Important scoring caveat: the `optimal` and `optimalityHint` fields in the question data are not used by the scoring path. The Reverse String heuristic can reward less appropriate code because it penalizes `while` loops. These values are source-pattern hints, not time/space complexity analysis.

05 / BUILD, RUN, DEPLOY

Commands and environment

Install the project dependencies once. During local development, run the frontend and multiplayer service in separate terminal windows.

```
npm install

# Terminal 1: multiplayer WebSocket server
npm run dual-server

# Terminal 2: frontend development server
npm run dev

# Production static bundle only
npm run build

# Serve a built bundle locally (build first)
npm run preview
```

Command/config	Purpose
`npm run dev`	Starts Vite, usually at http://localhost:5173. Does not start the WebSocket server.
`npm run dual-server`	Starts the Node service at port 3001 (or Render's `PORT`). Required for dual rooms only.
`npm run build`	Creates the static `dist/` bundle. It exits; it is not a long-running web server.
`VITE_DUAL_SERVER_URL`	Optional WebSocket URL. Blank means same host on port 3001; production should use the Render `wss://.../dual-ws` address.

Deployment checklist

- Render service: repository root, Node runtime, `npm install`, `npm run dual-server`, health path `/health`.

- Vercel frontend: build with `npm run build`; publish `dist`; define `VITE_DUAL_SERVER_URL` before build and redeploy.

- Remote players need HTTPS frontend plus reachable WSS room service. A local Render free service may sleep and cold-start.

- No external evaluator or API key is configured in the current project. Dual-mode evaluation is performed in each player's browser.

> **Same Wi-Fi/LAN**
>
> Run `npm run dual-server` and `npm run dev -- --host 0.0.0.0`. Both devices visit `http://:5173`; allow inbound TCP ports 5173 and 3001.

06 / REVIEW AND FILE MAP

What is solid, what to improve

Area	Current state / recommendation
Bundle size	Build succeeds, but the main minified JavaScript is about 3.87 MB and Monaco language workers add large chunks. Load Monaco/dual mode lazily and reduce bundled language workers.
Single-player code	`new Function` runs on the UI thread every animation frame. A runaway script can freeze the page. Move it to a worker or document that only trusted code should be run.
Dual worker trust	The worker has a 2-second timeout and no DOM, but browser workers can still use network APIs. This is not a secure multi-tenant code sandbox.
Fair scoring	Correctness and score are computed by the browser and trusted by the server. A competitive deployment should validate submissions and scores server-side.
Efficiency signal	Regex heuristics are easy to game and one current heuristic is misleading. Use them as optional coaching hints, not as a decisive winner score.
Room durability	Room state is memory-only. Add persistent storage if rooms must survive server restarts or scale across multiple server instances.
Lint	Oxlint exits 0 with warnings, including unused App imports, refs read during render, and incomplete effect dependency lists. Clean these up before expanding features.

Source map

Path	Role
`src/App.jsx`	Lobby and single-player orchestration.
`src/components/GameCanvas.jsx`	Claw machine simulation and API.
`src/components/BowlingCanvas.jsx`	Bowling physics, pin simulation and API.
`src/components/DualRoom.jsx`	Room UI, live editor sync, timer, submission and score UI.
`src/dualChallenges.js` + `codeRunner.worker.js`	Problem definitions, test cases, score heuristic and JavaScript worker.
`server/dualServer.js`	HTTP health route and in-memory WebSocket room server.
`vite.config.js`, `render.yaml`	Frontend tooling and Render deployment configuration.

Dependency note: `apify-client` and `@monaco-editor/react` are listed in `package.json`, but the reviewed source imports neither. Confirm whether they are still needed before removing them.